

OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

The Crop Recommendation System is a Machine Learning-based web application that helps farmers choose the most suitable crop based on soil nutrients and environmental conditions. The application predicts the best crop by analyzing important agricultural parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, soil pH, and rainfall.

The system is developed using Python, Flask, HTML, CSS, and Machine Learning. A Logistic Regression model is trained on agricultural datasets and saved using Pickle. The Flask backend receives user inputs, processes them through the trained model, and returns the recommended crop instantly.

The application provides an easy-to-use interface where users can enter field conditions and receive accurate crop recommendations. This helps improve agricultural productivity, reduce crop failure, and support data-driven farming decisions.

Scenarios

Scenario 1:

A farmer enters the following soil details:

- Nitrogen: 90
- Phosphorus: 42
- Potassium: 43
- Temperature: 21°C
- Humidity: 82%
- pH: 6.5
- Rainfall: 202 mm

The system predicts **Rice** as the most suitable crop based on the given environmental conditions.

Scenario 2:

A farmer from a dry region enters:

- Nitrogen: 70
- Phosphorus: 50
- Potassium: 40
- Temperature: 30°C
- Humidity: 55%
- pH: 7.1
- Rainfall: 75 mm

The application recommends **Maize**, which grows well under these climatic conditions.

Scenario 3:

A farmer provides:

- Nitrogen: 35
- Phosphorus: 25
- Potassium: 30
- Temperature: 27°C
- Humidity: 65%
- pH: 6.8
- Rainfall: 110 mm

The system predicts **Cotton** as the best crop and displays the recommendation within seconds.

Technical Architecture

Description

The Crop Recommendation System follows a **three-layer architecture**.

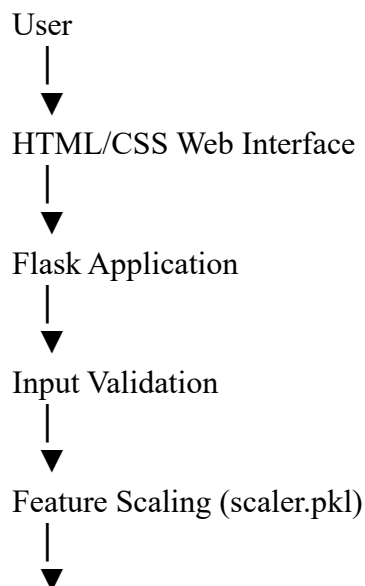
The **Presentation Layer** consists of HTML, CSS, and JavaScript pages that collect user inputs through an interactive web interface.

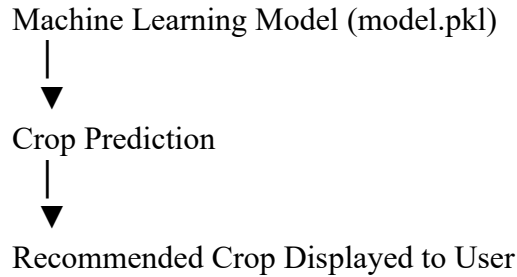
The **Application Layer** is developed using Flask. It validates the user inputs, loads the trained machine learning model (model.pkl) and scaler (scaler.pkl), processes the data, and performs prediction.

The **Machine Learning Layer** contains the trained Logistic Regression model built using Scikit-learn. The model predicts the most suitable crop based on the input parameters.

The prediction result is sent back to the Flask application and displayed on the webpage.

Architecture Flow





Pre-requisites

Before developing and running the Crop Recommendation System, ensure that the following software, tools, libraries, and technologies are installed and understood:

- **Python 3.10 or later** – Used for implementing the machine learning model and backend logic.
- **Flask Framework** – Used to develop the web application and handle client-server communication.
- **Machine Learning Concepts** – Basic understanding of classification algorithms, model training, and prediction.
- **Scikit-learn (sklearn)** – Used for data preprocessing, model training, evaluation, and prediction.
- **Pandas** – Used for loading, cleaning, and manipulating the agricultural dataset.
- **NumPy** – Used for numerical computations and array operations.
- **Matplotlib & Seaborn** – Used for Exploratory Data Analysis (EDA) and data visualization.
- **Jupyter Notebook** – Used for data preprocessing, model building, and performance evaluation.
- **HTML5** – Used to create the web pages and user interface.
- **CSS3** – Used for styling and designing the web application.
- **JavaScript** – Used to improve user interaction and form validation.
- **Visual Studio Code (VS Code)** – Used as the development environment for coding and debugging.
- **Git & GitHub** – Used for version control and project management.
- **Pickle** – Used to save and load the trained machine learning model (model.pkl) and scaler (scaler.pkl).
- **Web Browser (Google Chrome/Microsoft Edge)** – Used to access and test the Flask web application.
- **Windows/Linux Operating System** – Supported platform for development and deployment.

Required Python Libraries

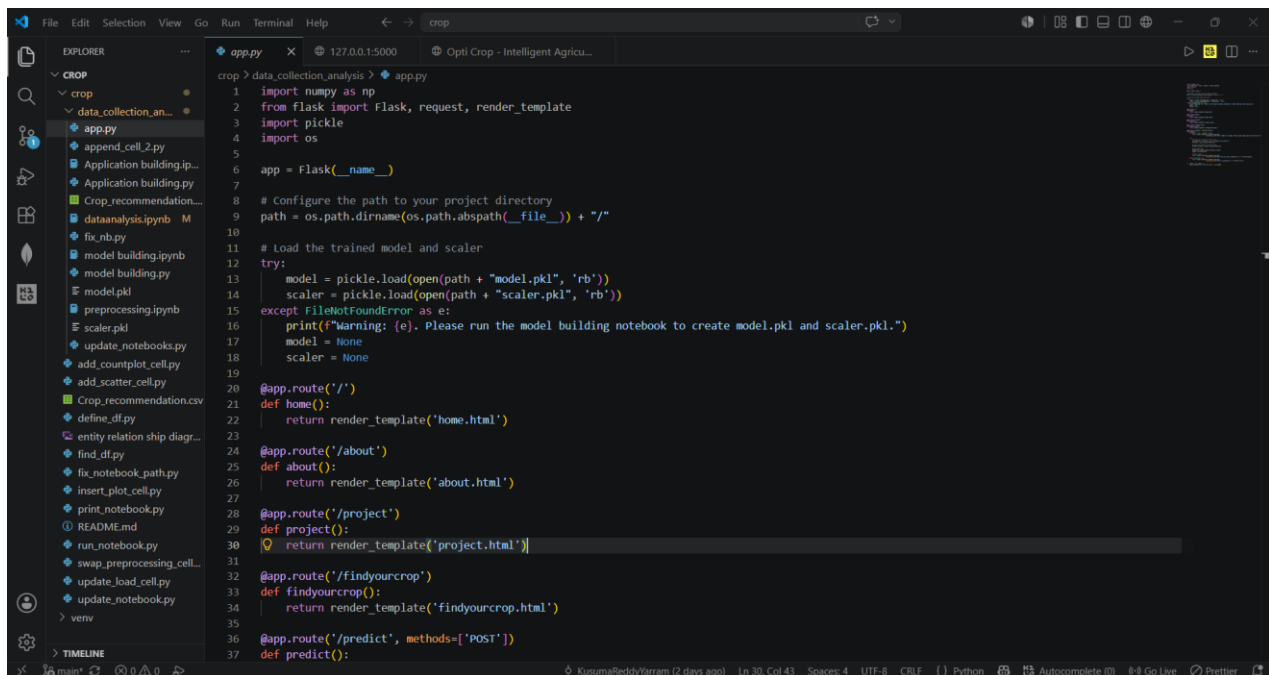
pip install flask
pip install pandas
pip install numpy
pip install scikit-learn
pip install matplotlib
pip install seaborn
pip install joblib

Milestone 1: Project Setup

Step 1: Project Structure

Content:

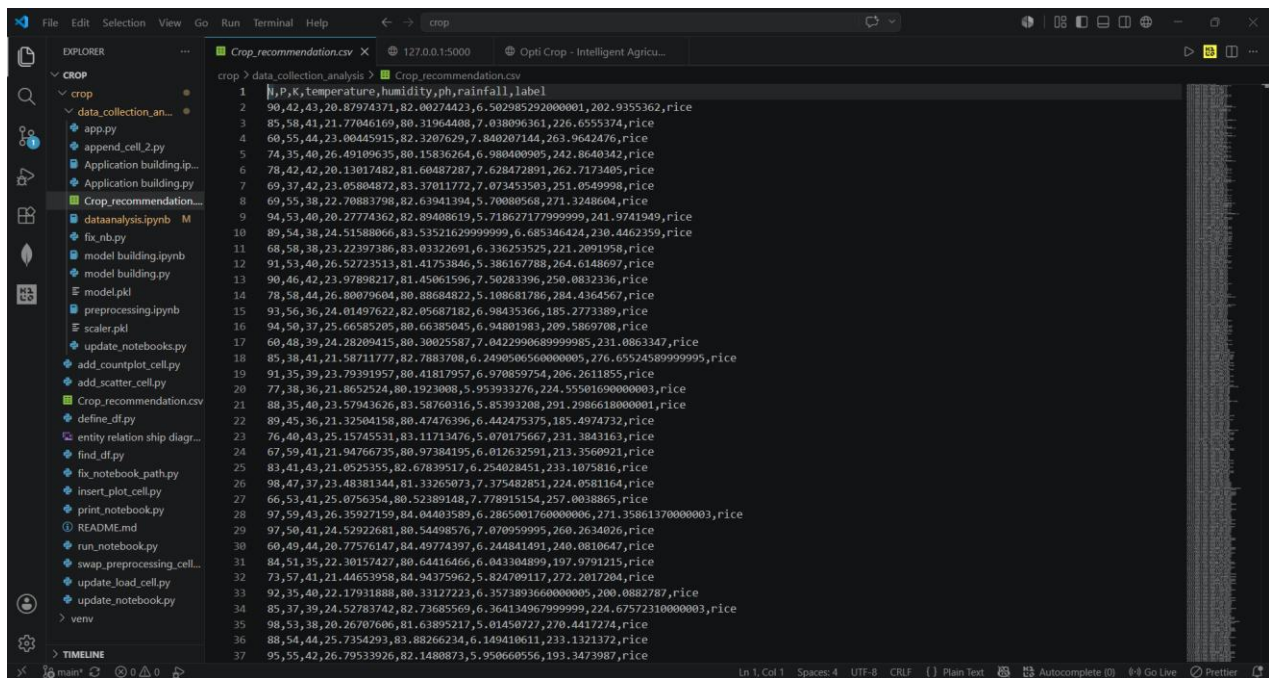
The Crop Recommendation System project was created using Python and Flask. The project contains the machine learning model, dataset, templates, and application files. The folder structure is organized to separate the backend, frontend, and machine learning resources.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'crop' and 'data_collection_analysis', and files like 'app.py', 'data_collection_analysis.py', 'model.pkl', 'scaler.pkl', 'update_notebooks.py', 'add_countplot_cell.py', 'add_scatter_cell.py', 'Crop_recommendation.csv', 'define_df.py', 'entity relation ship diag...', 'find_df.py', 'fix_notebook_path.py', 'insert_plot_cell.py', 'print_notebook.py', 'README.md', 'run_notebook.py', 'swap_preprocessing_cell...', 'update_load_cell.py', 'update_notebook.py', and 'venv'. The code editor shows the content of 'app.py'.

```
1 import numpy as np
2 from flask import Flask, request, render_template
3 import pickle
4 import os
5
6 app = Flask(__name__)
7
8 # Configure the path to your project directory
9 path = os.path.dirname(os.path.abspath(__file__)) + "/"
10
11 # Load the trained model and scaler
12 try:
13     model = pickle.load(open(path + "model.pkl", 'rb'))
14     scaler = pickle.load(open(path + "scaler.pkl", 'rb'))
15 except FileNotFoundError as e:
16     print(f"Warning: {e}. Please run the model building notebook to create model.pkl and scaler.pkl.")
17     model = None
18     scaler = None
19
20 @app.route('/')
21 def home():
22     return render_template('home.html')
23
24 @app.route('/about')
25 def about():
26     return render_template('about.html')
27
28 @app.route('/project')
29 def project():
30     return render_template('project.html')
31
32 @app.route('/findyourcrop')
33 def findyourcrop():
34     return render_template('findyourcrop.html')
35
36 @app.route('/predict', methods=['POST'])
37 def predict():
```

The crop dataset contains soil nutrients and environmental parameters such as Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, Rainfall, and Crop Label. This dataset is used to train the machine learning model.



```

1  N,P,K,temperature,humidity,ph,rainfall,label
2  90,42,43,20.87974371,82.00274423,6.502985292000001,202.9355362,rice
3  85,58,41,21.77046169,80.31964408,7.038096361,226.6555374,rice
4  60,55,44,23.00445015,82.3207629,7.840207144,263.9642476,rice
5  74,35,40,26.49109635,80.15836264,6.980408905,242.8640342,rice
6  78,42,42,20.13617482,81.60487287,7.628472891,262.7173405,rice
7  69,37,42,23.05804872,83.37011772,7.073453583,251.0549998,rice
8  69,55,38,22.70883798,82.63041394,5.70080968,271.3248604,rice
9  94,53,40,20.27774362,82.89408619,5.718627177999999,241.9741049,rice
10 80,54,38,24.51880066,83.535216299999999,6.685346424,230.4462359,rice
11 68,58,38,23.22397386,83.0322691,6.33623525,221.2091958,rice
12 91,53,40,26.52723513,81.41753846,5.386167788,264.6148697,rice
13 90,46,42,23.97898217,81.45061596,7.50281396,250.0832336,rice
14 78,58,44,26.80079604,80.88684822,5.108681786,284.4364567,rice
15 93,56,36,24.01497622,82.05687182,6.98435366,185.2773389,rice
16 94,50,37,25.66585205,80.66385045,6.94801983,209.5869708,rice
17 60,48,39,24.28209415,80.30025587,7.04229906899999985,231.0863347,rice
18 85,38,41,21.58711777,82.7883708,6.2490506560000005,276.655245899999995,rice
19 91,35,39,23.79391957,80.41817957,6.970859754,206.2611855,rice
20 77,38,36,21.8652524,80.1923008,5.953933276,224.55501690000003,rice
21 88,35,40,23.57943626,83.58760316,5.85393208,291.29866180000001,rice
22 89,45,36,21.32504158,80.47476396,6.442475375,185.4974732,rice
23 76,40,43,25.15745531,83.11713476,5.070175667,231.3843163,rice
24 67,59,41,21.94766735,80.97384195,6.012632591,213.3560921,rice
25 83,41,43,21.0525355,82.67839517,6.254028451,233.1075816,rice
26 98,47,37,23.48381344,81.33265073,7.375482851,224.0581164,rice
27 66,53,41,25.0756354,80.52389148,7.778915154,257.0038865,rice
28 97,59,43,26.35927159,84.04403589,6.2865001760000006,271.35861370000003,rice
29 97,50,41,24.52922681,80.54409576,7.070959995,260.2634026,rice
30 60,49,44,20.77576147,84.49774397,6.244841491,240.0810647,rice
31 84,51,35,22.30157427,80.64416466,6.043304899,197.9791215,rice
32 73,57,41,21.44653958,84.94375962,5.824709117,272.2017204,rice
33 92,35,40,22.17931888,80.33127223,6.3573893660000005,200.0882787,rice
34 85,37,39,24.52783742,82.73685569,6.364134967999999,224.67572310000003,rice
35 98,53,38,20.26707606,81.63895217,5.01450727,270.4417274,rice
36 88,54,44,25.7354293,83.88266234,6.149410611,233.1321372,rice
37 95,55,42,26.79533926,82.1488873,5.950660556,193.3473987,rice
  
```

Step 2: Model Building

Content:

The Logistic Regression algorithm is used to train the crop recommendation model. After training, the model is saved as model.pkl and the scaler is saved as scaler.pkl for future predictions.

Milestone 3: Flask Application Development

Step 1: Backend Development

Content:

The Flask application loads the trained machine learning model and provides routes for the Home, About, Project, and Crop Prediction pages.

Code

from flask import Flask

```
app = Flask(__name__)
```

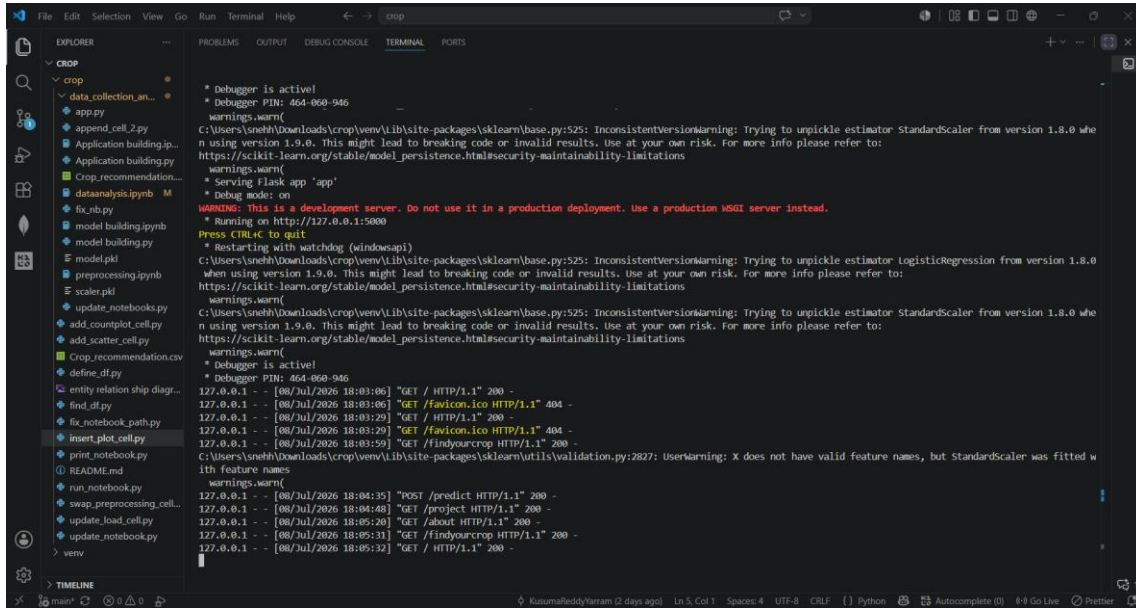
```
@app.route("/")
```

```
def home():
```

```
    return render_template("home.html")
```

```
@app.route("/predict", methods=["POST"])
def predict():
```

Step 2: Run the Application



```
File Edit Selection View Go Run Terminal Help
crop

* Debugger is active!
* Debugger PIN: 464-868-946
warnings.warn(
C:\Users\snehh\Downloads\crop\venv\Lib\site-packages\sklearn\base.py:525: InconsistentVersionWarning: Trying to unpickle estimator StandardScaler from version 1.8.0 whe
n using version 1.9.0. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL-C to quit
* Restarting with watchdog (windowsapi)
C:\Users\snehh\Downloads\crop\venv\Lib\site-packages\sklearn\base.py:525: InconsistentVersionWarning: Trying to unpickle estimator LogisticRegression from version 1.8.0
when using version 1.9.0. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
C:\Users\snehh\Downloads\crop\venv\Lib\site-packages\sklearn\base.py:525: InconsistentVersionWarning: Trying to unpickle estimator StandardScaler from version 1.8.0 whe
n using version 1.9.0. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
* Debugger is active!
* Debugger PIN: 464-868-946
127.0.0.1 - - [08/Jul/2026 18:03:06] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [08/Jul/2026 18:03:06] "GET /Favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [08/Jul/2026 18:03:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [08/Jul/2026 18:03:29] "GET /Favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [08/Jul/2026 18:03:59] "GET /findyourcrop HTTP/1.1" 200 -
C:\Users\snehh\Downloads\crop\venv\Lib\site-packages\sklearn\utils\validation.py:2827: UserWarning: X does not have valid feature names, but StandardScaler was fitted w
ith feature names
warnings.warn(
127.0.0.1 - - [08/Jul/2026 18:04:35] "POST /predict HTTP/1.1" 200 -
127.0.0.1 - - [08/Jul/2026 18:04:46] "GET /project HTTP/1.1" 200 -
127.0.0.1 - - [08/Jul/2026 18:05:20] "GET /about HTTP/1.1" 200 -
127.0.0.1 - - [08/Jul/2026 18:05:31] "GET /findyourcrop HTTP/1.1" 200 -
127.0.0.1 - - [08/Jul/2026 18:05:32] "GET / HTTP/1.1" 200 -

> TIMELINE
main
```

Milestone 4: Website Interface

Home Page

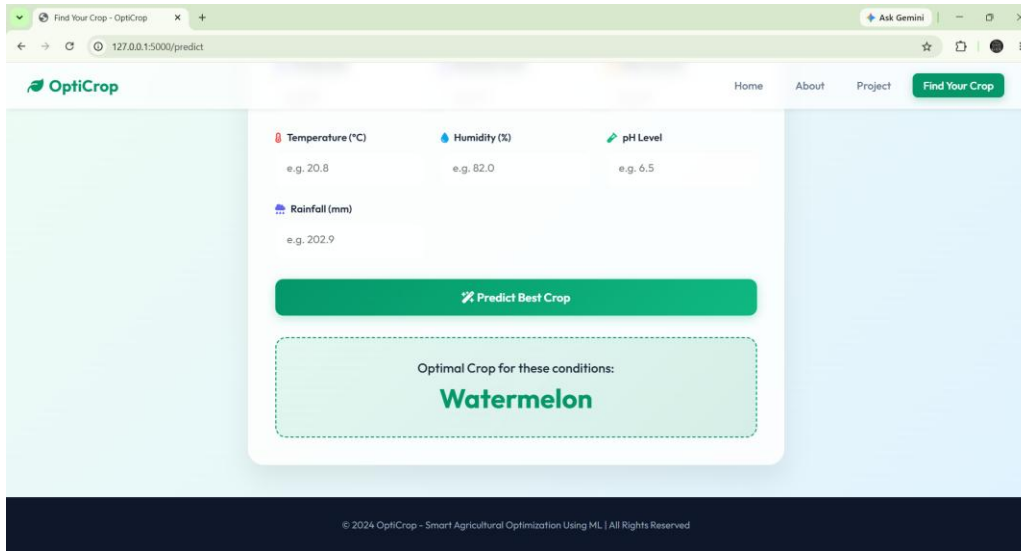
Description:

The Home page introduces the OptiCrop Crop Recommendation System. It provides navigation to different sections and explains how machine learning helps farmers choose suitable crops.



Find Your Crop Page

The Find Your Crop page allows users to enter soil nutrients and environmental parameters such as Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall. These values are submitted to the prediction model.



Find Your Crop - OptiCrop

127.0.0.1:5000/predict

OptiCrop

Home About Project Find Your Crop

Temperature (°C) Humidity (%) pH Level

e.g. 20.8 e.g. 82.0 e.g. 6.5

Rainfall (mm)

e.g. 202.9

Predict Best Crop

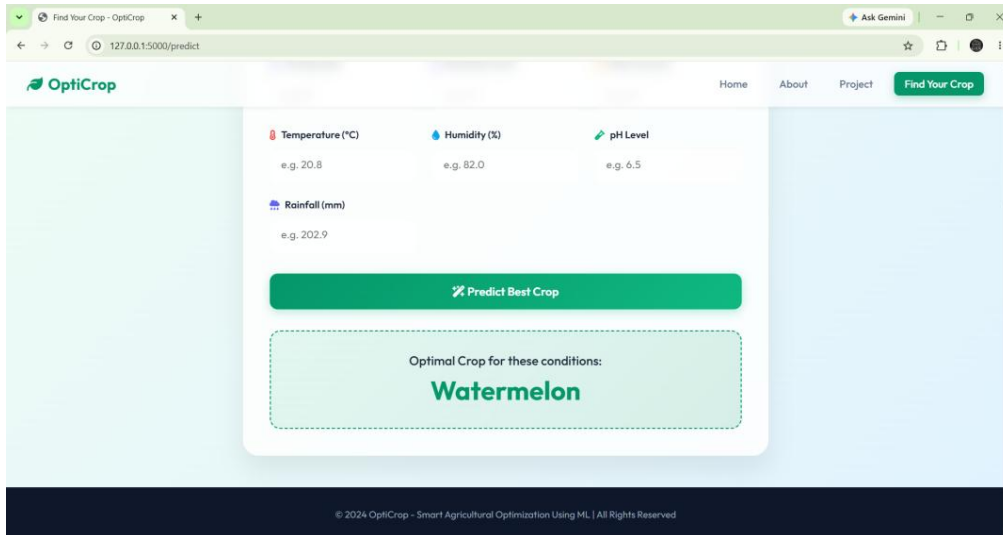
Optimal Crop for these conditions:

Watermelon

© 2024 OptiCrop - Smart Agricultural Optimization Using ML | All Rights Reserved

Prediction Result

After entering the required values, the machine learning model predicts the most suitable crop and displays the recommendation instantly.



Find Your Crop - OptiCrop

127.0.0.1:5000/predict

OptiCrop

Home About Project Find Your Crop

Temperature (°C) Humidity (%) pH Level

e.g. 20.8 e.g. 82.0 e.g. 6.5

Rainfall (mm)

e.g. 202.9

Predict Best Crop

Optimal Crop for these conditions:

Watermelon

© 2024 OptiCrop - Smart Agricultural Optimization Using ML | All Rights Reserved